

Comprehensive Software Analysis and Design Report: Library Management System

This document consolidates Requirements Engineering concepts from Lecture 3 and applies them directly to the Library Management System project as a single comprehensive report suitable for GitHub documentation, class presentation, and technical review.

1. Problem Definition and System Scope

The Library Management System addresses inefficiencies in manual library operations including tracking borrowed books, managing returns, handling fines, managing reservations, and maintaining accurate inventory.

In scope: catalog search, member registration, borrowing, returns, reservations, fine management, librarian operations, administrative configuration, reporting.

Out of scope for current version: payment gateway integration, multi-branch synchronization, RFID automation, advanced recommendation engines.

2. Stakeholder Analysis

Primary stakeholders: System Administrator, Librarian, Member (Student).

Administrator requirements: configure policies, define fine rates, manage staff roles, review reports, maintain system-wide controls.

Librarian requirements: issue books, process returns, manage inventory, register members, handle overdue cases, support reservations.

Member requirements: search books, borrow books, reserve unavailable titles, view due dates, manage account obligations.

3. Functional Requirements

FR1 The system shall allow members to search the catalog by title, author, and genre.

FR2 The system shall validate member eligibility before borrowing.

FR3 The system shall create a transaction record for every borrowing event.

FR4 The system shall update book availability immediately after issue or return.

FR5 The system shall calculate overdue conditions and support fine processing.

FR6 The system shall allow reservations for unavailable books.

FR7 The system shall support reporting for borrowed books, overdue books, and inventory status.

4. Non-Functional Requirements

Performance: book search responses should complete within acceptable response thresholds.

Availability: the system should be available during operational hours with minimal downtime.

Security: role-based access control must restrict unauthorized actions.

Usability: librarians and members should perform core tasks with minimal training.

Reliability: transaction and inventory data must remain consistent after operations.

5. Use Cases and Core Workflows

Borrow Book Use Case: actor searches book, system checks availability, librarian validates request, system records transaction, inventory updates.

Return Book Use Case: actor returns book, system checks due date, system applies overdue logic, inventory updates, transaction closes.

Reserve Book Use Case: actor places hold when book unavailable, reservation recorded, member notified when available.

6. System Models and Design Artifacts

ER Model entities: Member, Book, Transaction, Reservation, Fine, Librarian.

Use Case Diagram models actor-system interactions.

Activity Diagrams model borrowing and return workflows including decision points.

Sequence perspective models interactions among UI, backend, and database.

7. Requirements Elicitation and Analysis

Methods applied: stakeholder interviews, scenario analysis, use-case modeling, requirements classification, prioritization.

Analysis addresses ambiguity, consistency, conflict resolution, and traceability.

8. Requirements Validation

Validation criteria: validity, consistency, completeness, realism, verifiability.

Validation methods: walkthrough reviews, prototype-based checks, scenario test cases for borrow/return/reservation paths.

9. Requirements Management and Change

Changes shall be tracked through requirement identifiers, traceability links, and impact analysis.

Potential future changes: digital memberships, automated fines, notification services, expanded reporting.

10. Implementation Mapping (Prototype Architecture)

Frontend: React user interface for search, borrow, return.

Backend: Node.js/Express API for business logic.

Database: MySQL relational model supporting transactions and integrity constraints.

11. Risks and Constraints

Risks include ambiguous requirements, incomplete stakeholder input, and requirement changes during development.

Constraints include academic project scope, limited implementation time, and simplified prototype assumptions.

12. Conclusion

The project aligns requirements engineering theory with a concrete system design. It demonstrates stakeholder-driven analysis, structured requirements, system modeling, validation planning, and a path toward scalable implementation.